Module 2

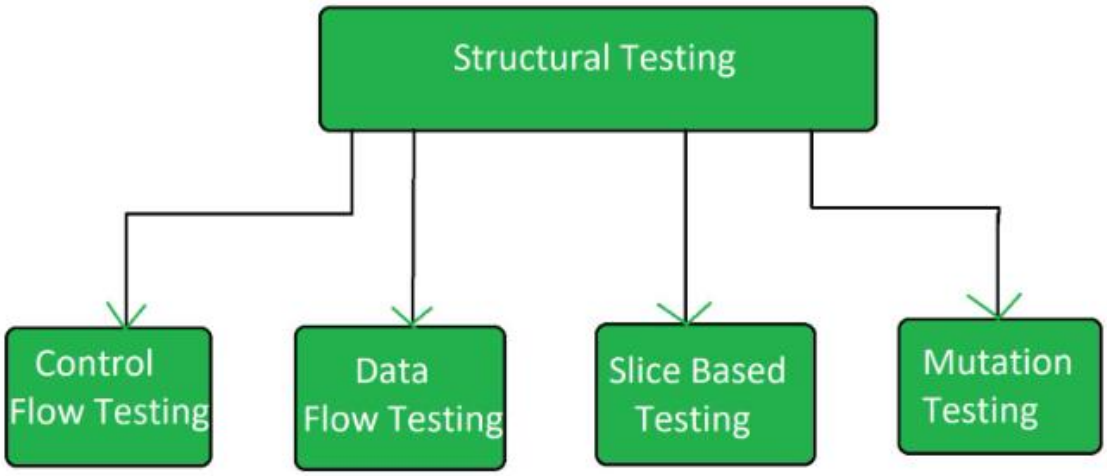
Whitebox Testing:
The primary goal of white box testing is to focus on the flow of inputs and outputs through the software and to strengthen the security of the software.
The white box testing contains various tests, which are as follows:

- Path testing
- Loop testing
- Condition testing
- Testing based on the memory perspective
- Test performance of the program

Structural Testing:
The structure-based testing technique is also known as the 'white-box' or 'glass-box' testing technique because the testers require knowledge of how the software is implemented and works.

Structural testing is basically related to the internal design and implementation of the software i.e. it involves the development team members in the testing team. It basically tests different aspects of the software according to its types.

<u>Types of Structural testing</u>



Control Flow Testing:
Control flow testing is a type of structural testing that uses the program's control flow as a model. The entire code, design, and structure of the software have to be known for this type of testing. Often this type of testing is used by the developers to test their own code and implementation. This method is used to test the logic of the code so that the required result can be obtained.
Data Flow Testing:

It uses the control flow graph to explore the unreasonable things that can happen to data. The detection of data flow anomalies is based on the associations between values and variables. Without being initialized usage of variables. Initialized variables are not used once.

Slice-Based Testing:

It was initially proposed by Weiser and Gallagher for software maintenance. It is useful for software debugging, software maintenance, program understanding, and quantification of functional cohesion. It divides the program into different slices and tests that slice which can majorly affect the entire software.

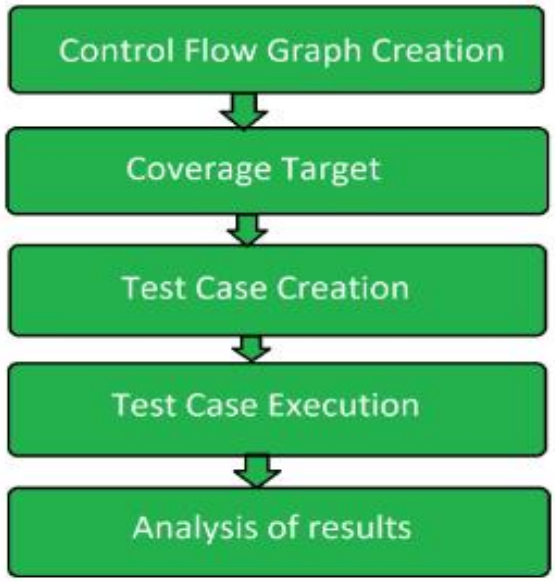
Mutation Testing:

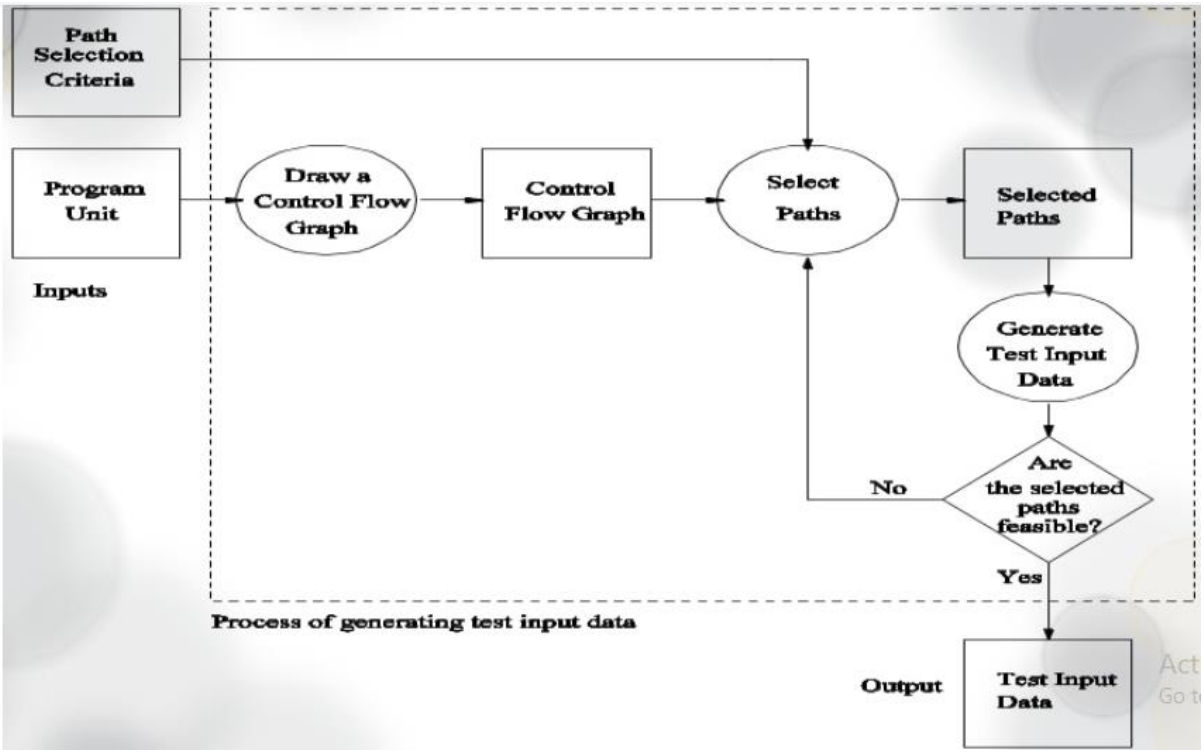
Mutation Testing is a type of Software Testing that is performed to design new software tests and also evaluate the quality of already existing software tests. Mutation testing is related to modifying a program in small ways. It focuses to help the tester develop effective tests or locate weaknesses in the test data used for the program.

Control Flow Testing

Control flow testing is a structural testing strategy that uses a program's control flow as a model.

Following are the steps involved in the process of control flow testing:





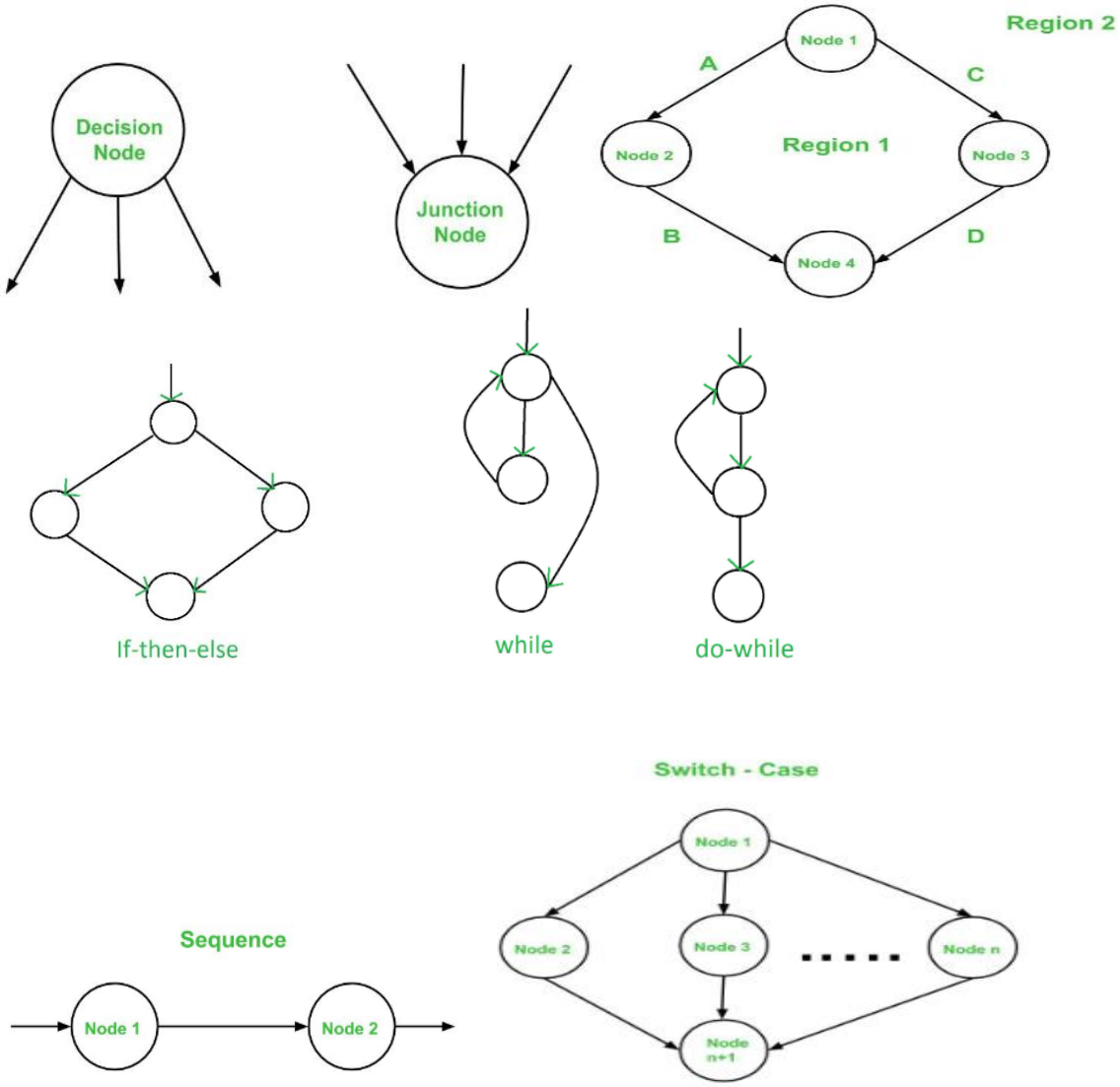
Node: It represents one or more procedural statements.
Edges or Links: They represent the flow of control in a program.
Decision node: A node with more than one arrow leaving is called a decision node.
Junction Node: A node with more than one arrow entering it is called a junction.
Regions: Areas bounded by edges and nodes are called regions.

The Cyclomatic complexity can be measured by knowing the number of independent paths in a program's source code, the formula is

V(G)=E-N+2P
OR
V(G)= D+1
OR
V(G)=R+1

Where,
E=edges, N=nodes, P= components (no. of exit points),
D=Decision node, R= enclosed regions.

Basics on control graph creation



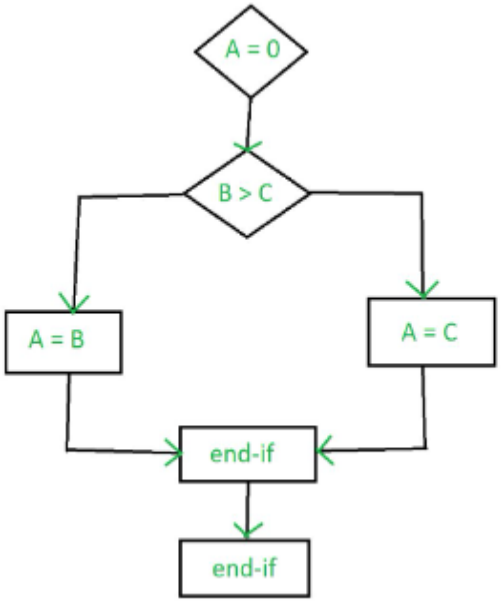
Basis Path Testing is a white-box testing technique based on the control structure of a program or a module. Basis path testing is a technique of selecting the paths in the control flow graph, that provide a basis set of execution paths through the program or module.

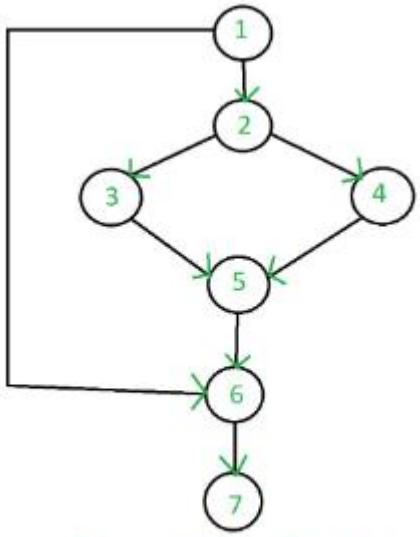
To design test cases using this technique, four steps are followed:

1. Construct the Control Flow Graph
2. Compute the Cyclomatic Complexity of the Graph
3. Identify the Independent Paths
4. Design Test cases from Independent Paths

Example 1:

```
if  A = 10 then
  if B > C
     A = B
  else A = C
  endif
  endif
print A, B, C
```





Control Flow Graph

Cyclomatic Complexity:

1. V(G) = E-N+2P
   Where, E→ No. of edges
   N→ No. of nodes
   P → No. of exit points

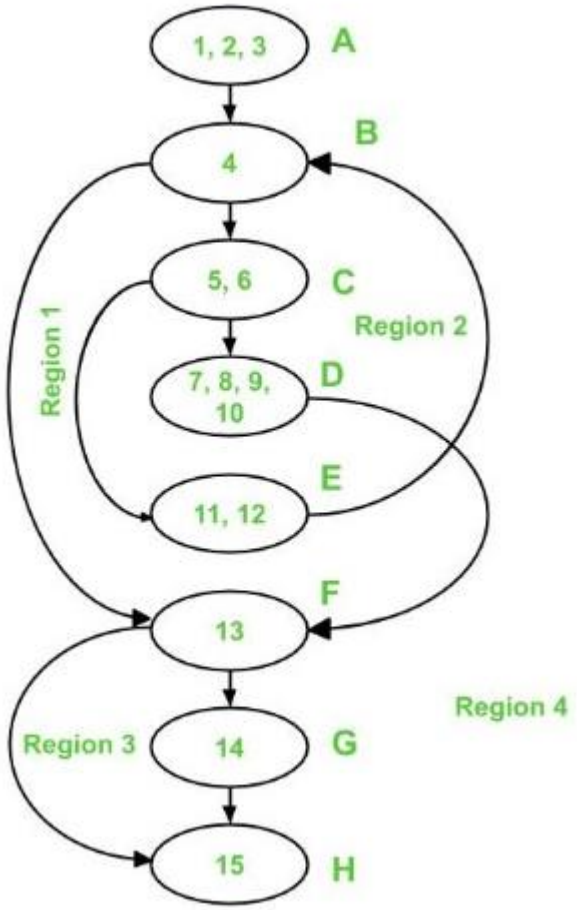
   V(G) = 8-7+2(1)
   = 1+2
   = 3

2. V(G) = P+1
   Where P→ Decision-making nodes
   = 2+1
   = 3

3. V(G) = R+1
   Where, R→ No. of regions
   = 2+1
   =3

Example 2: Program to find whether a given number is prime or not.

```
int main()
{
    int n, index;
1    cout << "Enter a number: " <> n;
3    index = 2;
4    while (index <= n - 1)
5    {
6        if (n % index == 0)
7        {
8            cout << "It is not a prime number" << endl;
9            break;
10        }
11        index++;
12    }
13    if (index == n)
14        cout << "It is a prime number" << endl;
15  } // end main
```



Cyclomatic Complexity:

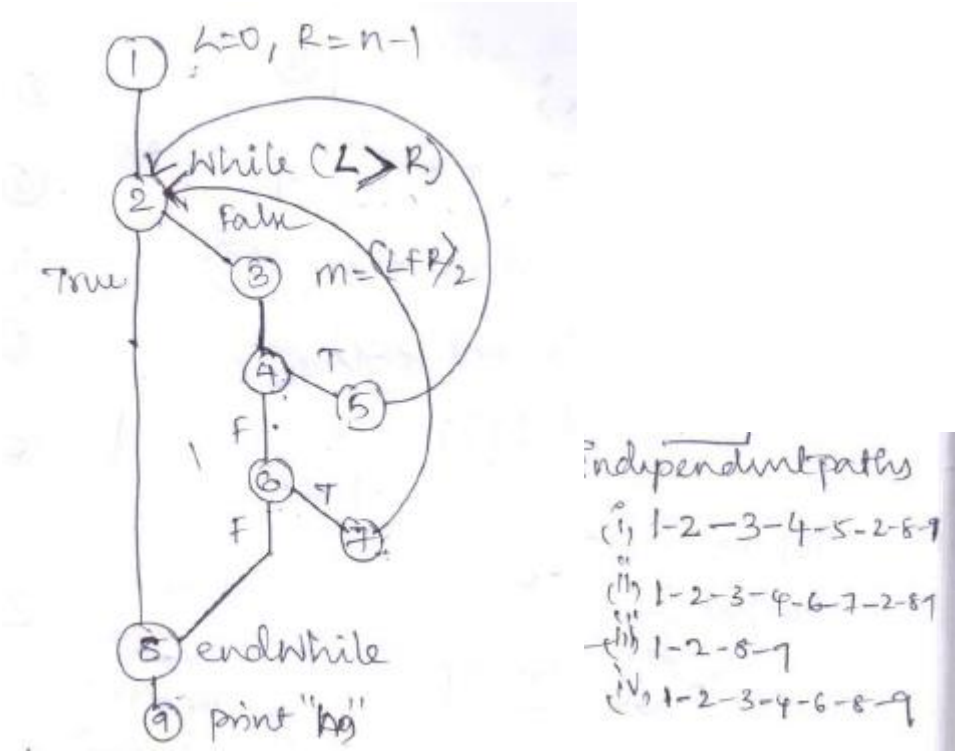
1. V(G) = E-N+2P
   E=10, N=8, P=1
   V(G) = 10-8+2*1
   = 2+2
   = 4

2. V(G) = P+1
   = 3+1 (Node 1,6,13)
   = 4

3. V(G) = R+1
   = 3+1
   =4

Example 3 : Binary search Algorithm

1.set L to 0 & R to n-1
2.while L>R then search terminates unsuccessfully
3.let m=(L+R)/2
4.if k<=m
5.L=0, R=m-1,goto2
6.elseif K>m
7.L=m+1, R=n-1,goto(2)
8.end while
9.K=m, return m



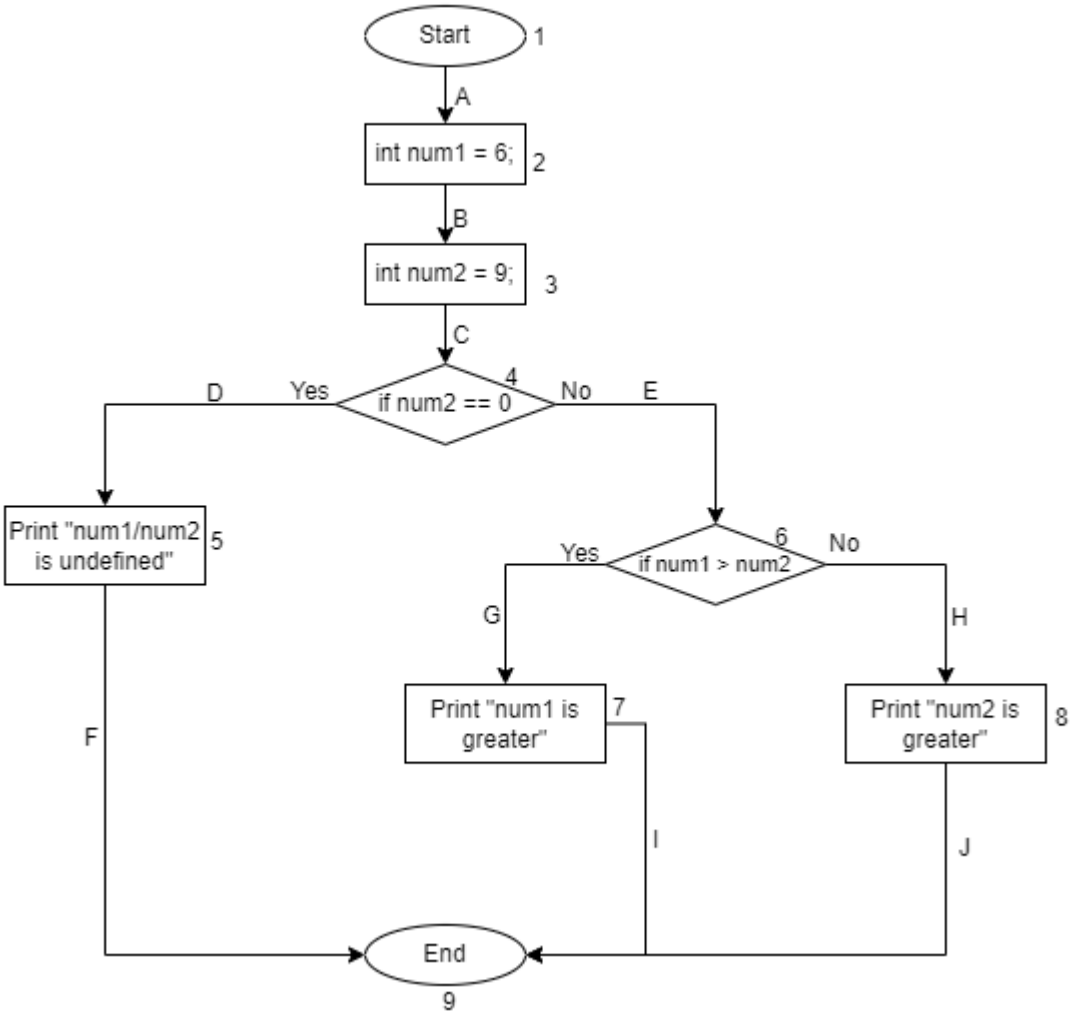
V(G) = E-N+2P
= 11-9+2(1)
= 2+2
= 4

V(G) = P+1
= 3+1 (Node 2,4,6)

V(G) = R+1
= 3+1
= 4

Example 4: Greatest among two numbers

```
int num1 = 6;
int num2 = 9;
if(num2 == 0){
  cout<<"num1/num2 is undefined"<<endl;
}else{
  if(num1 > num2){
    cout<<"num1 is greater"<<endl;
  }else{
    cout<<"num2 is greater"<<endl;
  }
}
```



*Cyclomatic complexity = E - N + 2*P*

*Cyclomatic complexity = 10 - 9 + 2*1*

Cyclomatic complexity = 3

- *Path 1*: 1A-2B-3C-4D-5F-9
- *Path 2*: 1A-2B-3C-4E-6G-7I-9
- *Path 3*: 1A-2B-3C-4E-6H-8J-9

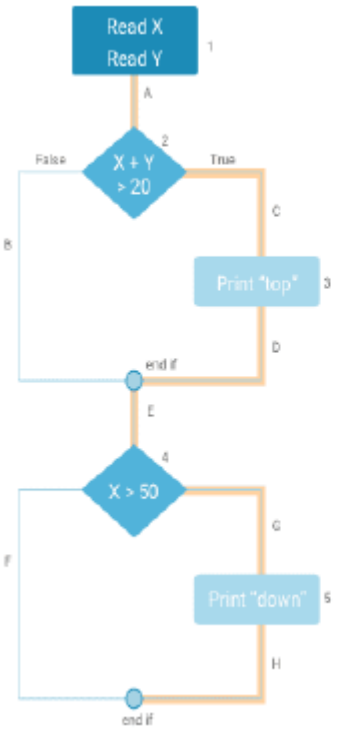
Path	Input values
Path 1: 1A-2B-3C-4D-5F-9	• num1 = 9 • num2 = 0
Path 2: 1A-2B-3C-4E-6G-7I-9	• num1 = 4 • num2 = 2
Path 3: 1A-2B-3C-4E-6H-8J-9	• num1 = 6 • num2 = 8

Test Coverage Metrics

It is a technique to ensure that your tests are testing your code or how much of your code you exercised by running the test.

Test Coverage Metrics
1. Statement coverage:
2. Branch coverage:
3. Path coverage:

Statement coverage is a white box testing technique, which involves the execution of all the statements at least once in the source code.

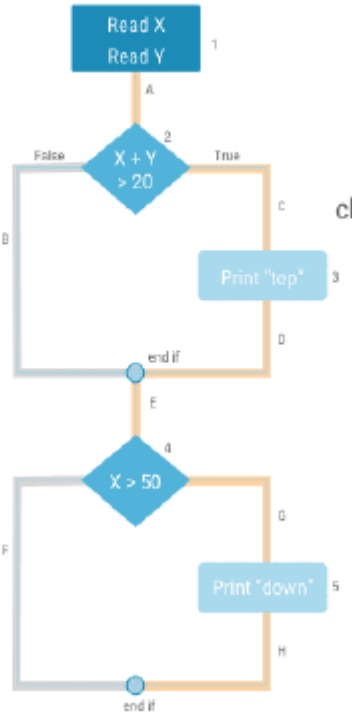


Path that covers all the statements in the flowchart:

1A - 2C - 3D - E - 4G - 5H

Branch coverage is a requirement that for each branch in the program (e.g., if statements, loops), each branch has been executed at least once during testing.

Branch 2:
checking "false" condition

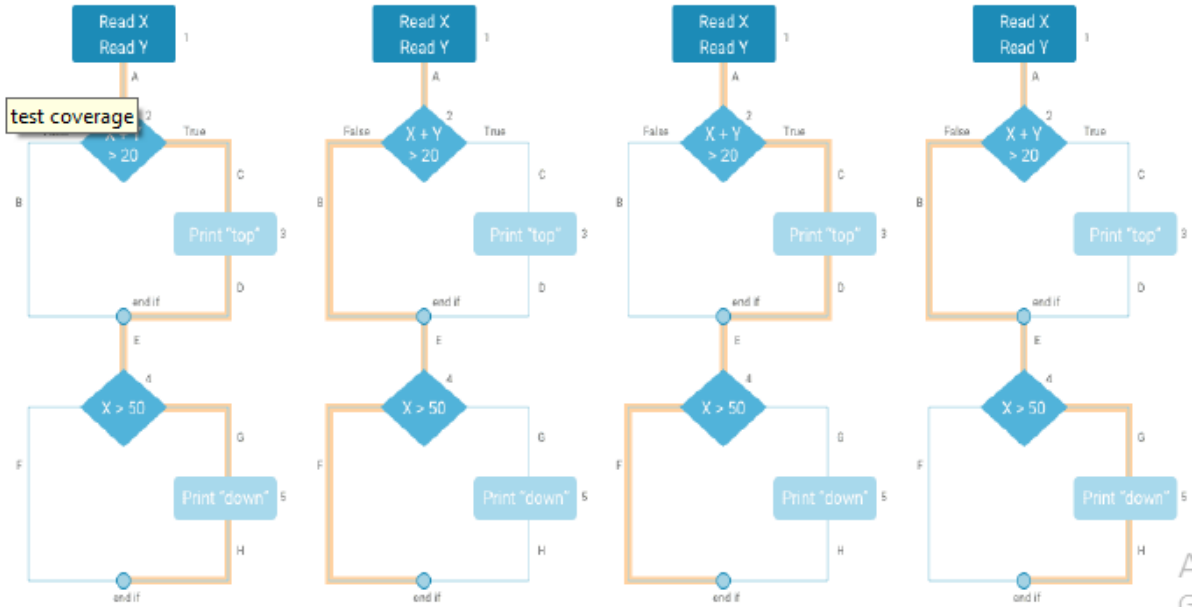


Branch 1:
checking "true" condition

Branch coverage is 2
1A - 2C - 3D - E- 4G -5H
1A - 2B - E - 4F

Path coverage refers to designing test cases such that all linearly independent paths in the program are executed at least once. A linearly independent path can be defined in terms of what's called a control flow graph of an application.

- 1A-2B-E-4F
- 1A-2B-E-4G-5H
- 1A-2C-3D-E-4G-5H
- 1A-2C-3D-E-4F



Data Flow testing

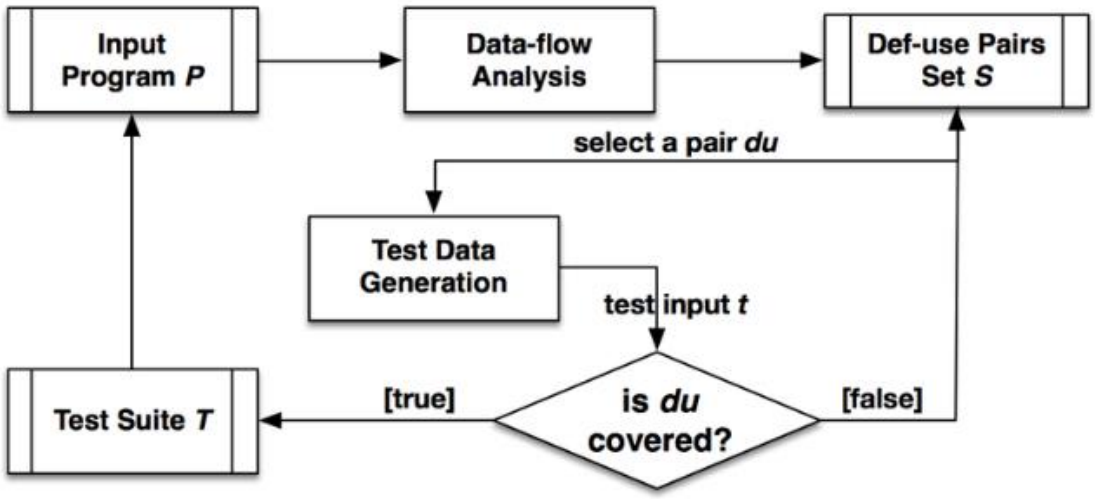
Data flow testing is a type of structural testing used to analyze the flow of data in the program. It is a method that is used to find the test paths of a program according to the locations of definitions and uses of variables in the program.

It is concerned with:

- Statements where variables receive values,
- Statements where these values are used or referenced.

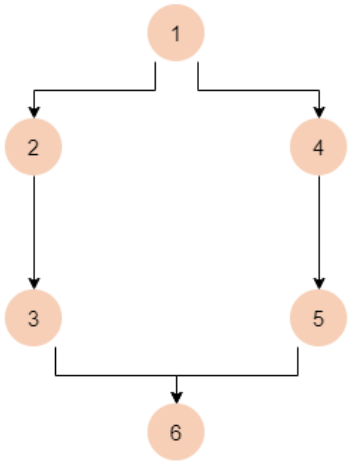
Data flow testing can be done using one of the following two techniques:

- Control flow graph
- Making associations between data definition and usage



Control flow graph example

```
1. input(x)
2. if(x>5)
3.     z = x + 10
4. else
5.     z = x - 5
6. print("Value of Z: ", z)
```



Variable Name	Defined At	Used At
x	1	2
z	3, 5	6

Making associations

In this technique, we make associations between two kinds of statements:

- Where variables are defined
- Where those variables are used

```
1. input(x)
2. if(x>5)
3.     z = x + 10
4. else
5.     z = x - 5
6. print("Value of Z: ", z)
```

For the above snippet of pseudo-code, we will make the following associations:

- (1, (2,t), x): for the true case of the IF statement in line 2
- (1, (2,f), x): for the false case of IF statement in line 2
- (1, 3, x): variable x is being used in line 3 to define the value of z
- (1, 5, x): variable x is being used in line 5 to define the value of z
- (3, 6, z): variable z is being used in line 6, which is defined in line 3
- (5, 6, z): variable z is being used in line 6, which is defined in line 5

The first two associations are for the IF statement on line 2. One association is made if the condition is true, and the other is for the false case.
Now, there are two types of uses of a variable:

- **predicate use**: the use of a variable is called p-use. Its value is used to decide the flow of the program, e.g., line 2.
- **computational use**: the use of a variable is called c-use when its value is used compute another variable or the output, e.g., line 3.

After the associations are made, these associations can be divided into these groups:

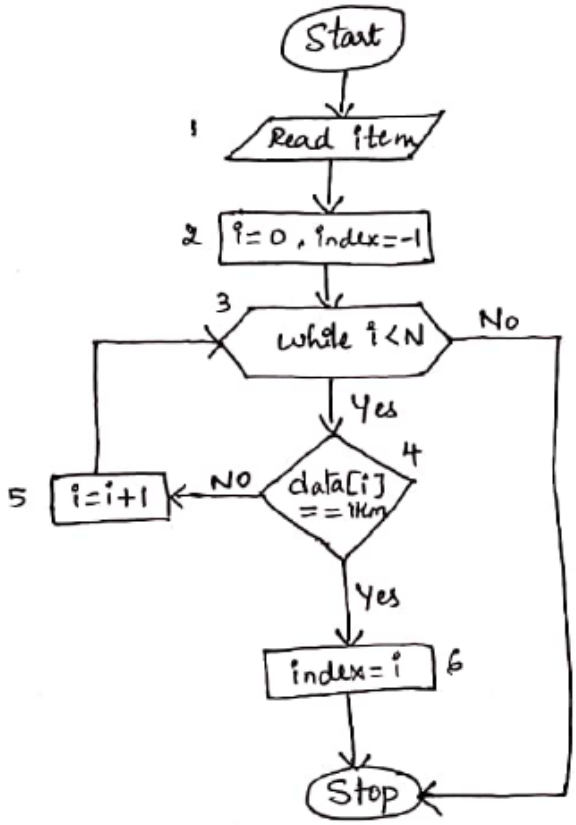
- All definitions coverage
- All p-use coverage
- All c-use coverage
- All p-use, some c-use coverage
- All c-use, some p-use coverage
- All use coverage

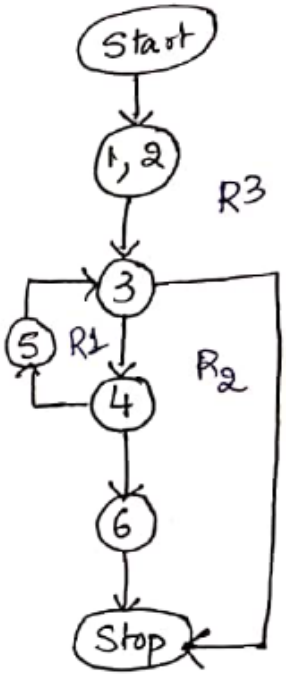
Once the associations are divided into these groups, the tester makes test cases and examines each point. The statements and variables found to be extra are removed from the code.

Data Flow Testing Coverage:

The coverage of data flow in terms of "sub-paths" and "complete paths" may be categorized under the following types:
All definition coverage: Covers "sub-paths" from each definition to some of their respective use.
All definition-C use coverage: "sub-paths" from each definition to all their respective C use.
All definition-P use coverage: "sub-paths" from each definition to all their respective P use.
All use coverage: Coverage of "sub-paths" from each definition to every respective use irrespective of types.
All definition use coverage: Coverage of "simple sub-paths" from each definition to every respective use.

Flowchart for Linear Search Algorithm.





Control graph

Variables	Defined	used.
item	1	4
i	2	3, 4, 6
index	2, 6	—
N	—	3

du-pairs

(1, 4)

(2, 3) (2, 4)

(2, 6)

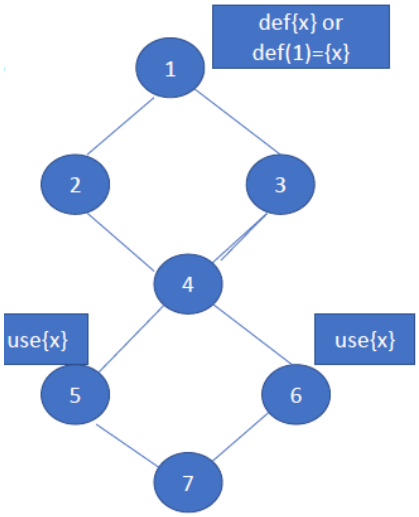
du-paths

{1, 2, 3, 4}

{2, 3}

{2, 3, 4} {2, 3, 4, 6}

Example for different dataflow coverage



All definitions coverage: Consider at least one path where the variable is defined and used
1-2-4-5
All use coverage: Consider the paths where the variable is defined and used
(All uses should be covered)
1-2-4-5
1-3-4-6
All du coverage: Paths of du
•1-2-4-5
•1-2-4-6
•1-3-4-5
•1-3-4-6

REGRESSION TESTING

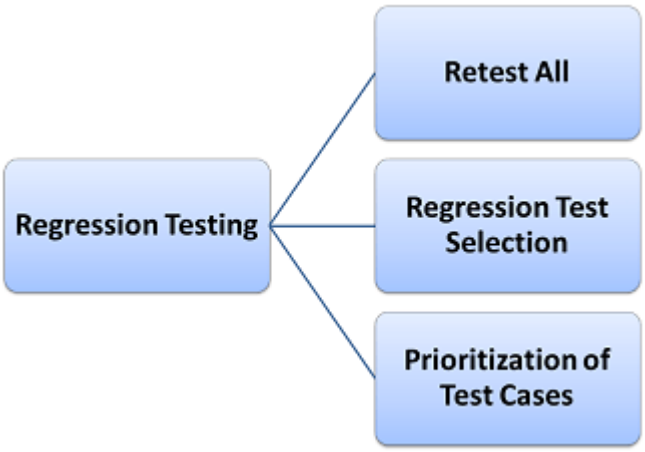
Regression testing is a software testing practice that ensures an application still functions as expected after any code changes, updates, or improvements.

corrective regression testing: It refers to regression testing of a program obtained by making corrections to the previous versions.
progressive regression testing: It refers to regression testing of a program obtained by adding new features.

Typically, regression testing is applied under these circumstances:

- A new requirement is added to an existing feature
- A new feature or functionality is added
- The codebase is fixed to solve defects
- The source code is optimized to improve performance
- Patch fixes are added
- Changes in configuration

Regression testing can be performed using the following techniques:



1. Re-test All:

Re-Test is one of the approaches to do regression testing. In this approach, all the test case suits should be re-executed. Here we can define re-test as when a test fails, and we determine the cause of the failure is a software fault. If the fault is reported, we can expect a new version of the software in which the defect is fixed. In this case, we will need to execute the test again to confirm that the fault is fixed. This is known as re-testing. Some will refer to this as confirmation testing.

The re-test is very expensive, as it requires enormous time and resources.

2. Regression test Selection:

In this technique, a selected test-case suit will execute rather than an entire test-case suit. The selected test case suits are divided into two cases

- **Reusable test cases** can use in succeeding regression cycles.
- **Obsolete test cases** can't use in succeeding regression cycles.

3. Prioritization of test cases:

Prioritize the test case depending on business impact, and critical and frequent functionality used. The selection of test cases will reduce the regression test suite.

Need for Regression Testing:

•Regression testing is also needed when a subsystem is modified to generate a new version of an application.

•When one or more components of an application are modified the entire application must also be subject to regression testing.

•In some cases regression testing might be needed when the underlying hardware changes. In this case, regression testing is performed despite any change in the software.

•Regression testing can be applied in each phase of software development.

REGRESSION TEST PROCESS

Test revalidation refers to the task of checking which tests for P remain valid for P'. Revalidation is necessary to ensure that only tests that are applicable to P' are used during regression testing.

Test Selection: The identification of tests that traverse modified portions of P' is often referred to as test selection and sometimes as the regression-test selection (RTS) problem.

There are 2 techniques for test selection.

1. Test minimization

2. Test prioritization

Test minimization discards tests as seemingly redundant with respect to some criteria.

Test prioritization refers to the task of prioritizing tests based on some criteria. Test selection can be achieved by selecting a few tests from a prioritized list.

The test setup refers to the process by which the application under test is placed in its intended, or simulated, environment ready to receive data and able to transfer any desired output information. Test setup requires simulators as a replacement for real devices and software and hardware environmental setup.

Eg1: a heart simulator is used while testing a commonly used heart control device known as the pacemaker

Eg2: The test setup process and the setup for automobile engine control software are quite different from that of a cellphone.

Test sequencing often becomes important for an application with an internal state that is continuously running.

Eg: Banking software, web service, and engine controller are examples of such applications.

Test execution: Once the testing infrastructure has been set up, tests selected, revalidated, and sequenced, it is time to execute them.

•This task is often automated using a generic or a special-purpose tool.

•General purpose tools are available to run regression tests for applications such as web service

•Embedded systems, due to their unique hardware requirements, often require special-purpose tools that input a test suite and automatically run the application

Output Comparison:

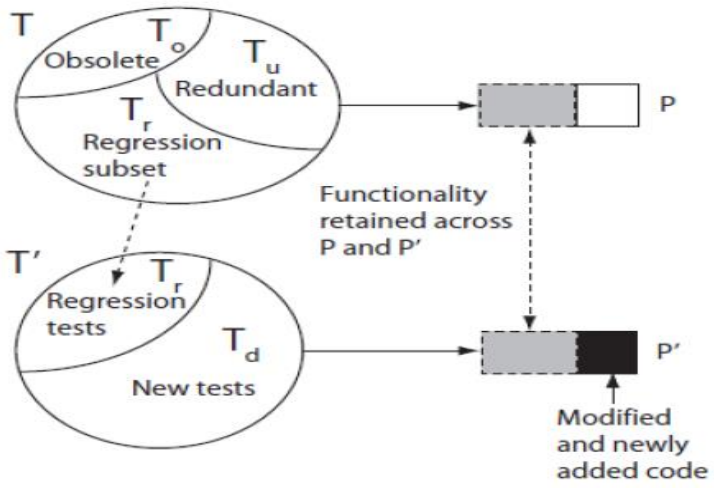
•Each test needs verification. This is done automatically using the test execution tool

•However, this might not be a simple process, especially in embedded systems.

•Sometimes for embedded systems we can't use generic tools

• goal for test execution is to measure an application's performance.

•Eg: one might want to know how many requests per second can be processed by a web service



TEST SELECTION FOR REGRESSION TESTING

Techniques to select tests in regression testing

1. Test All
We use T'=T-To to test all strategies
Disadvantage: Time consumption

2. Random Selection
Tests are selected randomly from the set T-To. The tester decides how many tests to select depending on the level of confidence required and the available time
Disadvantage:
Some of the sampled tests might bear no relationship to the modified code

3. Selecting Modification-Traversing Tests
We use methods to determine the desired subset and aim at obtaining a minimal regression test suite. Techniques that obtain a minimal regression-test suite without discarding any test that will traverse a modified statement are called safe RTS techniques
Advantage:
In a time crunch, testers execute only smaller no of tests
Disadvantage:
Sophisticated techniques to select modification-traversing tests require automation.
Impractical to apply for large commercial and complex systems

4. Test Minimization
It reduces the size of the regression test suite but discards the redundant tests
Disadvantage:
Tests discarded by the minimization algorithm must be reviewed carefully before discarding

5. Test Prioritization

A suitable metric is used to rank all the tests. A test with the highest rank has the highest priority

Eg: let R1, R2, and R3 require carried unchanged from P to P|. Rank-R2-R3-R1 and Td are new tests that need to be tested
{t1,t2,t3,t4,t5}-subset
t1-R1,
t2&t3-R2,
t4&t5-R3
We can prioritize as t2,t3,t4,t5,t1

6. Test selection using Execution Trace
Step 1: Given P and test set T, find the execution trace of P for each test in T.
Step 2: Extract test vectors from the execution traces for each node in the CFG of P
Step 3: Construct syntax trees for each node in the CFGs of P and P'. This step can be executed while constructing the CFGs of P and P's.
Step 4: Traverse the CFGs and determine the subset of T appropriate for regression testing of P'.

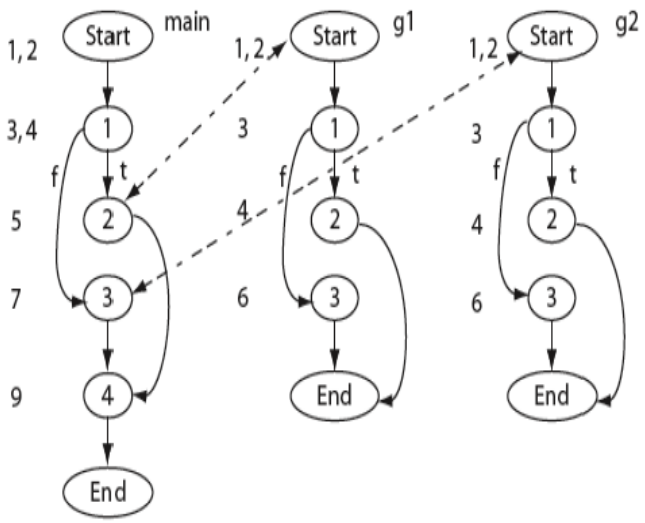
- Let G=(N, E) denote the CFG of program P
- N is a finite set of nodes
- E a finite set of edges connecting the nodes.
- Let Tno be the set of all valid tests for P'

```
1  main(){
2  int x,y,p;
3  input (x,y);
4  if (x<y)
5    p=g1(x,y);
6  else
7    p=g2(x,y);
8  endif
9  output (p);
10 end
11 }
```

```
1  int g1(int a, b){
2  int a,b;
3  if(a+ 1==b)
4    return(a*a);
5  else
6    return(b*b);
7  }
```

```
1  int g2 (int a, b){
2  int a,b;
3  if(a==(b+1))
4    return(b*b);
5  else
6    return(a*a);
7  }
```

Example program



CFG for example program

Test (t)	Execution trace ($trace(t)$)
t_1	main.Start, main.1, main.2, g1.Start, g1.1, g1.3, g1.End, main.2, main.4, main.End.
t_2	main.Start, main.1, main.3, g2.Start, g2.1, g2.2, g2.End, main.3, main.4, main.End.
t_3	main.Start, main.1, main.2, g1.Start, g1.1, g1.2, g1.End, main.2, main.4, main.End.

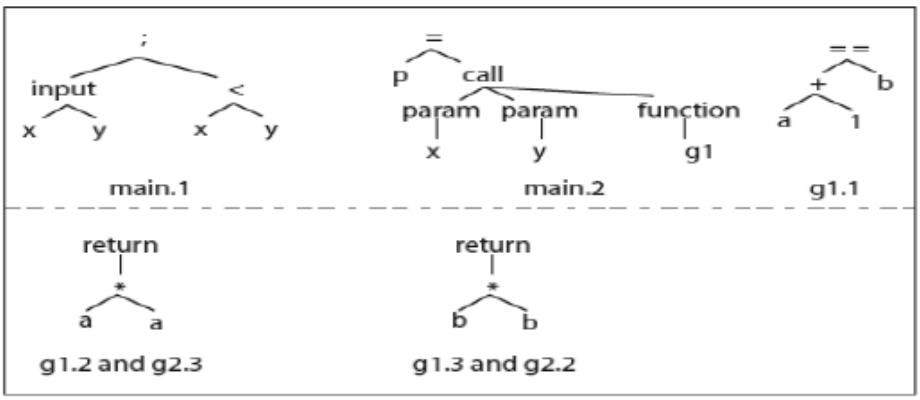
7. Test Vector and Syntax Trees

A test vector for node n, denoted by test(n), is the set of tests that traverse node n in the CFG

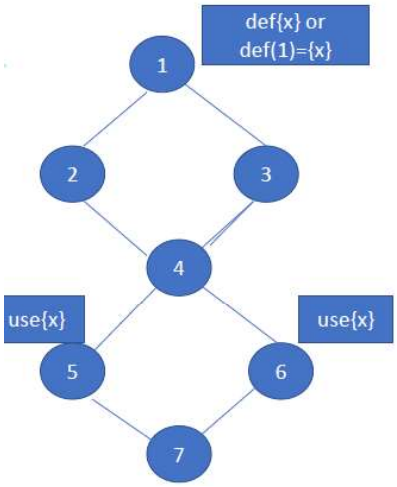
	Test vector ($test(n)$) for node n			
Function	**1**	**2**	**3**	**4**
main	t_1, t_2, t_3	t_1, t_3	t_2	$t_1, t_2\ t_3$
g1	t_1, t_3	t_3	t_1	–
g2	t_2	t_2	None	–

Syntax trees:

- A syntax tree is constructed for each node of CFG(P) and CFG(P').



Example for different dataflow coverage



All definitions coverage: Consider at least one path where the variable is defined and used
1-2-4-5
All use coverage: Consider the paths where the variable is defined and used
(All uses should be covered)
1-2-4-5
1-3-4-6
All du coverage: Paths of du
•1-2-4-5
•1-2-4-6
•1-3-4-5
•1-3-4-6

REGRESSION TESTING

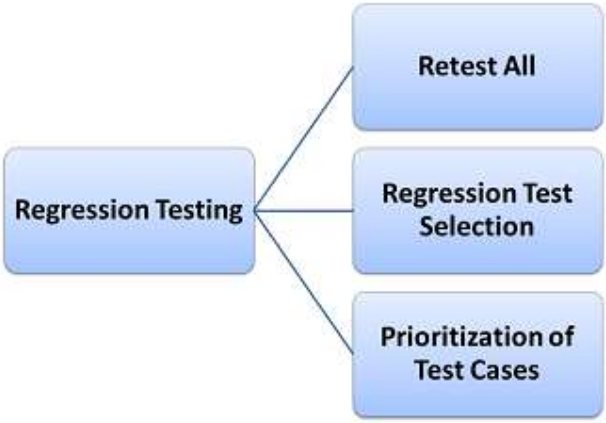
Regression testing is a software testing practice that ensures an application still functions as expected after any code changes, updates, or improvements.

corrective regression testing: It refers to regression testing of a program obtained by making corrections to the previous versions.
progressive regression testing: It refers to regression testing of a program obtained by adding new features.

Typically, regression testing is applied under these circumstances:

- A new requirement is added to an existing feature
- A new feature or functionality is added
- The codebase is fixed to solve defects
- The source code is optimized to improve performance
- Patch fixes are added
- Changes in configuration

Regression testing can be performed using the following techniques:



1. Re-test All:

Re-Test is one of the approaches to do regression testing. In this approach, all the test case suits should be re-executed. Here we can define re-test as when a test fails, and we determine the cause of the failure is a software fault. If the fault is reported, we can expect a new version of the software in which the defect is fixed. In this case, we will need to execute the test again to confirm that the fault is fixed. This is known as re-testing. Some will refer to this as confirmation testing.

The re-test is very expensive, as it requires enormous time and resources.

2. Regression test Selection:

In this technique, a selected test-case suit will execute rather than an entire test-case suit. The selected test case suits are divided into two cases

- **Reusable test cases** can use in succeeding regression cycles.
- **Obsolete test cases** can't use in succeeding regression cycles.

3. Prioritization of test cases:

Prioritize the test case depending on business impact, and critical and frequent functionality used. The selection of test cases will reduce the regression test suite.

Need for Regression Testing:

•Regression testing is also needed when a subsystem is modified to generate a new version of an application.

•When one or more components of an application are modified the entire application must also be subject to regression testing.

•In some cases regression testing might be needed when the underlying hardware changes. In this case, regression testing is performed despite any change in the software.

•Regression testing can be applied in each phase of software development.

REGRESSION TEST PROCESS

Test revalidation refers to the task of checking which tests for P remain valid for P'. Revalidation is necessary to ensure that only tests that are applicable to P' are used during regression testing.

Test Selection: The identification of tests that traverse modified portions of P' is often referred to as test selection and sometimes as the regression-test selection (RTS) problem.

There are 2 techniques for test selection.

1. Test minimization

2. Test prioritization

Test minimization discards tests as seemingly redundant with respect to some criteria.

Test prioritization refers to the task of prioritizing tests based on some criteria. Test selection can be achieved by selecting a few tests from a prioritized list.

The test setup refers to the process by which the application under test is placed in its intended, or simulated, environment ready to receive data and able to transfer any desired output information. Test setup requires simulators as a replacement for real devices and software and hardware environmental setup.

Eg1: a heart simulator is used while testing a commonly used heart control device known as the pacemaker

Eg2: The test setup process and the setup for automobile engine control software are quite different from that of a cellphone.

Test sequencing often becomes important for an application with an internal state that is continuously running.

Eg: Banking software, web service, and engine controller are examples of such applications.

Test execution: Once the testing infrastructure has been set up, tests selected, revalidated, and sequenced, it is time to execute them.

•This task is often automated using a generic or a special-purpose tool.

•General purpose tools are available to run regression tests for applications such as web service

•Embedded systems, due to their unique hardware requirements, often require special-purpose tools that input a test suite and automatically run the application

Output Comparison:

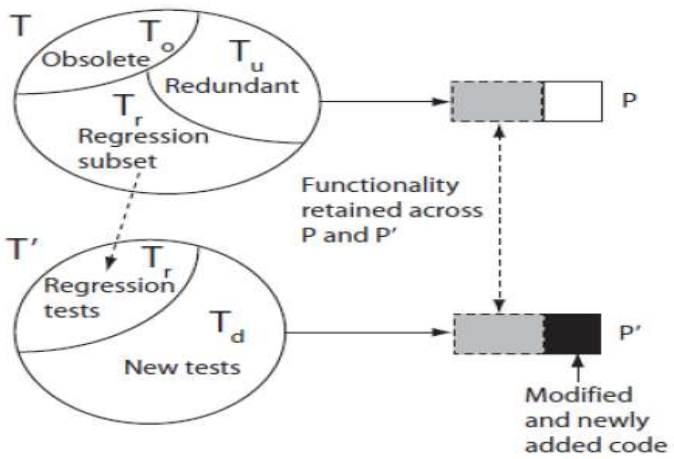
•Each test needs verification. This is done automatically using the test execution tool

•However, this might not be a simple process, especially in embedded systems.

•Sometimes for embedded systems we can't use generic tools

• goal for test execution is to measure an application's performance.

•Eg: one might want to know how many requests per second can be processed by a web service



TEST SELECTION FOR REGRESSION TESTING

Techniques to select tests in regression testing

1. Test All
We use T'=T-To to test all strategies
Disadvantage: Time consumption

2. Random Selection
Tests are selected randomly from the set T-To. The tester decides how many tests to select depending on the level of confidence required and the available time
Disadvantage:
Some of the sampled tests might bear no relationship to the modified code

3. Selecting Modification-Traversing Tests
We use methods to determine the desired subset and aim at obtaining a minimal regression test suite. Techniques that obtain a minimal regression-test suite without discarding any test that will traverse a modified statement are called safe RTS techniques
Advantage:
In a time crunch, testers execute only smaller no of tests
Disadvantage:
Sophisticated techniques to select modification-traversing tests require automation.
Impractical to apply for large commercial and complex systems

4. Test Minimization
It reduces the size of the regression test suite but discards the redundant tests
Disadvantage:
Tests discarded by the minimization algorithm must be reviewed carefully before discarding

5. Test Prioritization

A suitable metric is used to rank all the tests. A test with the highest rank has the highest priority

Eg: let R1, R2, and R3 require carried unchanged from P to P|. Rank-R2-R3-R1 and Td are new tests that need to be tested
{t1,t2,t3,t4,t5}-subset
t1-R1,
t2&t3-R2,
t4&t5-R3
We can prioritize as t2,t3,t4,t5,t1

6. Test selection using Execution Trace

Step 1: Given P and test set T, find the execution trace of P for each test in T.
Step 2: Extract test vectors from the execution traces for each node in the CFG of P
Step 3: Construct syntax trees for each node in the CFGs of P and P'. This step can be executed while constructing the CFGs of P and P's.
Step 4: Traverse the CFGs and determine the subset of T appropriate for regression testing of P'.

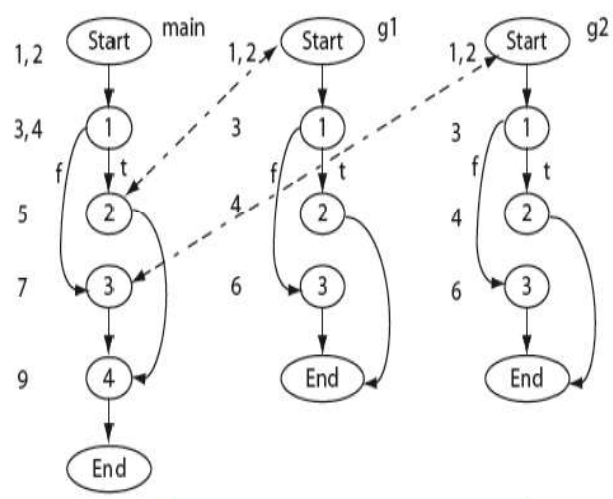
- Let G=(N, E) denote the CFG of program P
- N is a finite set of nodes
- E a finite set of edges connecting the nodes.
- Let Tno be the set of all valid tests for P'

```
1  main(){
2  int x,y,p;
3  input (x,y);
4  if (x<y)
5    p=g1(x,y);
6  else
7    p=g2(x,y);
8  endif
9  output (p);
10 end
11 }
```

```
1 int g1(int a, b){
2 int a,b;
3 if(a+ 1==b)
4   return(a*a);
5 else
6   return(b*b);
7 }
```

```
1 int g2 (int a, b){
2 int a,b;
3 if(a==(b+1))
4   return(b*b);
5 else
6   return(a*a);
7 }
```

Example program



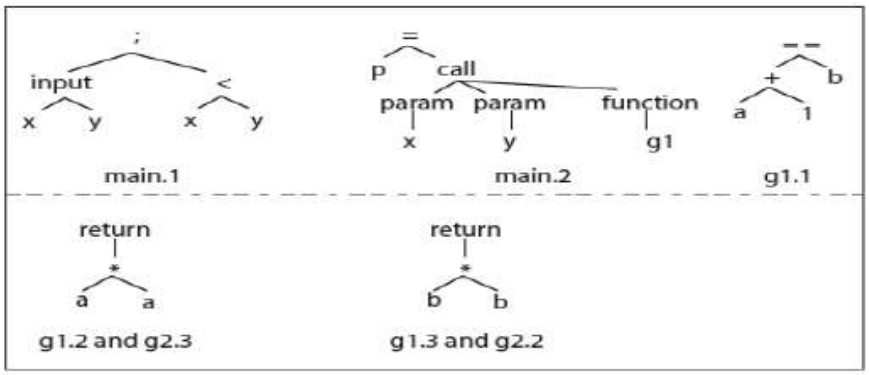
CFG for example program

Test (t)	Execution trace ($trace(t)$)
t_1	main.Start, main.1, main.2, g1.Start, g1.1, g1.3, g1.End, main.2, main.4, main.End.
t_2	main.Start, main.1, main.3, g2.Start, g2.1, g2.2, g2.End, main.3, main.4, main.End.
t_3	main.Start, main.1, main.2, g1.Start, g1.1, g1.2, g1.End, main.2, main.4, main.End.

7. Test Vector and Syntax Trees

A test vector for node n, denoted by test(n), is the set of tests that traverse node n in the CFG

	Test vector ($test(n)$) for node n			
Function	**1**	**2**	**3**	**4**
main	t_1, t_2, t_3	t_1, t_3	t_2	$t_1, t_2\ t_3$
g1	t_1, t_3	t_3	t_1	–
g2	t_2	t_2	None	–

Syntax trees:

- A syntax tree is constructed for each node of CFG(P) and CFG(P').